

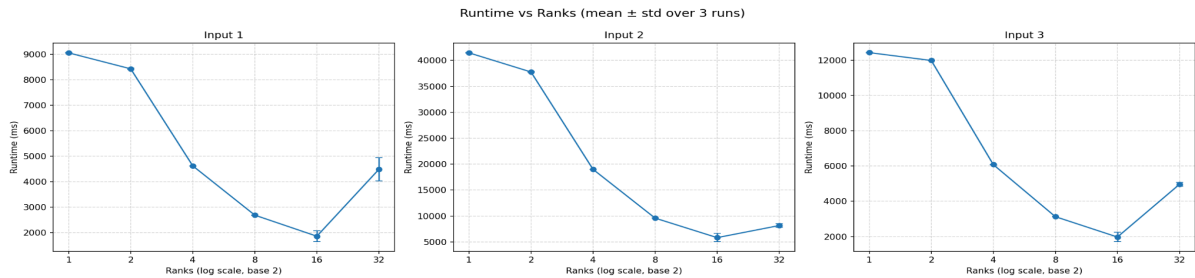
1. Brief Description

This project implements a parallel K-Nearest Neighbors (KNN) classifier using OpenMPI, where a brute-force approach computes squared Euclidean distances, maintains the top- k neighbors via a max-heap and performs majority voting. This was chosen for its simplicity, exactness and suitability for parallelisation. To improve efficiency, both dataset and queries are stored in flat, contiguous arrays which enhances cache locality and enables efficient MPI data transfers. Batched query processing is also used to evaluate multiple queries simultaneously, improving data reuse and reducing memory access overhead.

A key feature of the implementation is an adaptive parallelisation strategy that selects between two approaches based on workload characteristics. When the number of queries is small relative to the dataset size, the dataset is scattered across ranks while all queries are broadcast. After each rank computes its local top- k neighbors, partial results are merged via a two-level hierarchical tree reduction, first within each physical node using a binary-tree of blocking send/recv pairs that perform sorted two-pointer merges of neighbor lists, then across nodes where only node leaders participate. Conversely, when the number of queries is large, the full dataset is broadcast to all ranks and queries are scattered, allowing each rank to compute results independently without requiring merging. This strategy minimises communication overhead by reducing unnecessary data movement.

The implementation uses several OpenMPI constructs to support efficient communication. MPI_Bcast is used to distribute shared data such as the dataset or queries, while MPI_Scatterv and MPI_Gatherv handle uneven workload distribution and result collection. MPI_Allreduce ensures consistency of global parameters across ranks. Additionally, MPI_Comm_split_type and MPI_Comm_split are used to create node-local and leader communicators, enabling hierarchical communication. Custom tree merging is implemented using MPI_Send and MPI_Recv, as the top- k merging operation cannot be expressed using standard MPI reductions. As such, the program relies entirely on blocking MPI calls for synchronization, avoiding any explicit barrier or lock constructs.

1.1. Varying ranks and program performance



Performance improves up to 16 ranks but drops sharply at 32 because execution moves from fast intra-node communication to slower inter-node communication, introducing significant network latency; each node has 24 hardware threads, so up to 16 ranks still fits efficiently within a single node. The speedup from 1 to 2 ranks is minimal (~ 1.05 – $1.10\times$) since MPI overhead and serial merging dominate the limited reduction in computation.

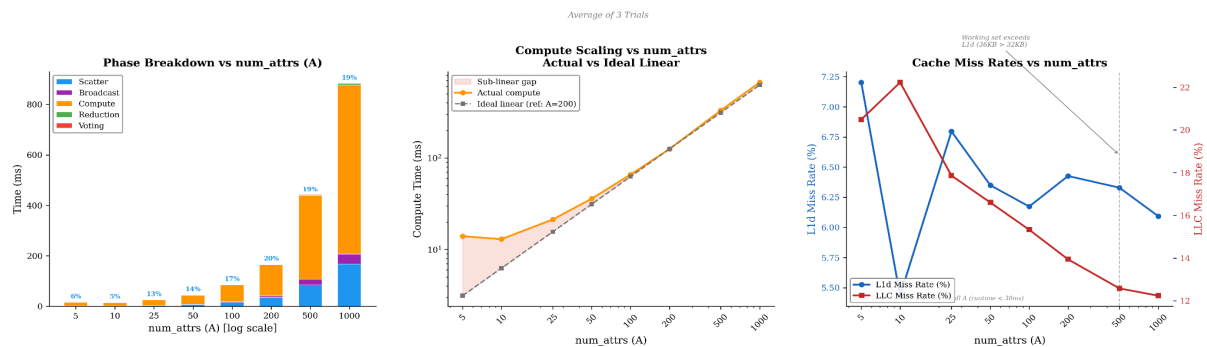
Near-linear scaling is observed at 4 and 8 ranks, where parallelism increases while communication remains efficient within a single node. Performance peaks at 16 ranks, beyond which inter-node communication and the growing cost of result merging outweigh further parallelisation benefits.

2. Factors affecting performance

2.1. How performance changes with parameters of input file

This section examines how two key input parameters affect the performance of the parallel MPI K-Nearest Neighbors program: dimensionality (`num_attrs`, A) and the number of nearest neighbors (k), each varied independently while holding other parameters fixed on a single w5-3423 node (Sapphire Rapids, 12C/24T, 24 MPI ranks) with results averaged over 3 trials.

2.1.1. How performance changes with varying dimensionality



Compute Scaling (Graph 2). For small A (5–25), compute time scales sub-linearly because the inner loop is too short for the CPU's out-of-order engine and SIMD pipeline to reach steady-state throughput. Here, fixed per-point overhead (heap operations, loop setup, pointer arithmetic) dominates. By $A \geq 200$, the loop is long enough for pipeline and vectorization costs to amortize, and scaling approaches the ideal linear rate.

Scatter and Communication (Graph 1). Scatter time scales linearly with A as expected from `MPI_Scatterv`, with effective bandwidth stabilizing around ~950 MB/s for larger transfers. The scatter fraction rises from 5% at $A=5$ to ~19% at $A \geq 200$, then levels off. Since both scatter and compute scale linearly for large A , their ratio converges to a constant. The program remains compute dominated at all A values.

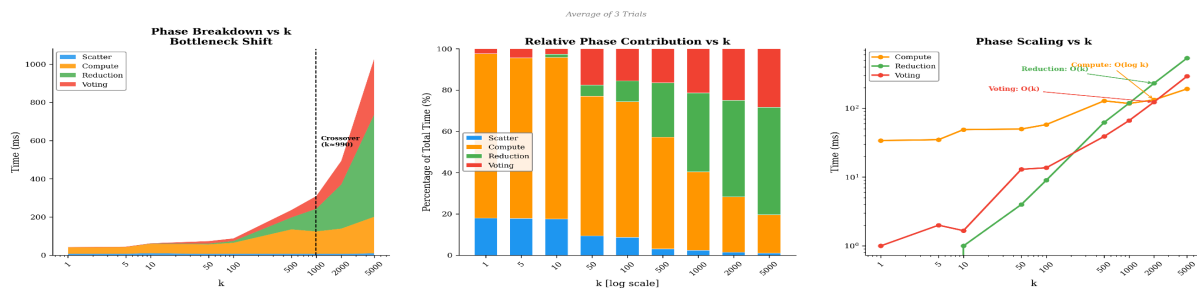
Cache Behaviour (Graph 3). At $A=500$, the hot working set per iteration is ~36 KB, exceeding the 32 KB L1d capacity but the miss rate remains stable at ~6%, which is unintuitive at first. However, this is because the access pattern is purely sequential. The code reads A contiguous doubles per data point, allowing the hardware prefetcher to stay ahead of demand regardless of working-set size. LLC miss rate actually decreases with A , as longer contiguous strides per data point make the streaming pattern easier for the prefetcher to predict.

2.1.2. How performance changes with varying k value (refer to graph below)

Bottleneck Shift (Graphs 1 and 2). The most significant finding is the complete inversion of the bottleneck. At $k=1$, the program is 74% compute-bound; at $k=5000$, compute drops to

18% while reduction+voting consumes 80%. The crossover occurs at $k \approx 1000$, where reduction time (119ms) matches compute (118ms).

Phase Scaling (Graph 3). Compute grows only as $O(\log k)$ because the distance calculation is identical regardless of k and the only added cost is heap maintenance, and most points are rejected early without touching the heap. Reduction scales as $O(k)$ due to the tree-based merge sending $Q \times k$ entries across $\lceil \log_2(24) \rceil = 5$ merge levels, with each level performing an $O(k)$ sorted merge per query. Voting also scales as $O(k)$, running serially on rank 0 with 5 million `std::map` lookups at $k=5000$, resulting in 292ms (28% of total time). Scatter remains flat across all k values, confirming it depends only on the fixed dataset size.

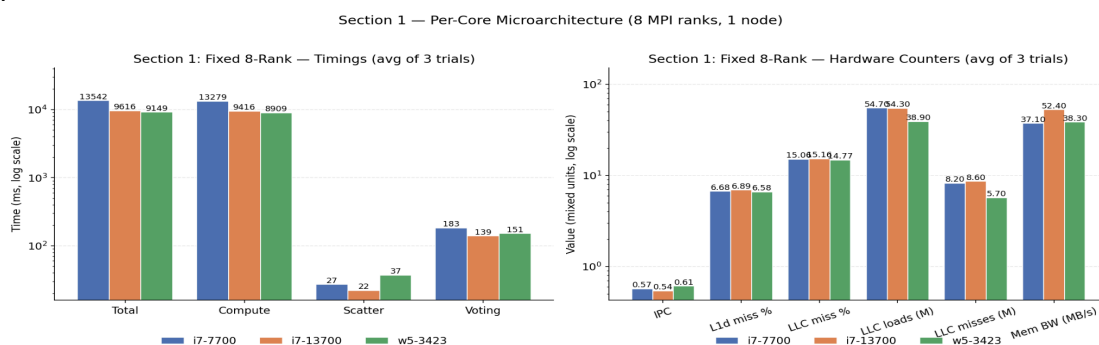


2.2. How performance changes with the type of machine it is run on

To evaluate how hardware affects the performance of the parallel k -NN classifier, three Intel CPUs spanning three microarchitecture generations were benchmarked: the Kaby Lake i7-7700, the Raptor Lake i7-13700, and the Sapphire Rapids w5-3423, all running the same -O3-compiled AVX2 binary against input2.in (50,000 data points, 40,000 queries, 50 attributes) averaged over 3 trials. 3 regimes were tested to isolate different hardware effects: a fixed 8-rank comparison that controls for parallelism and exposes per-core microarchitecture differences, a fully loaded single-node run (8/16/24 ranks) that measures peak single-node throughput, and a two-node run (16/32/48 ranks) that adds inter-node MPI communication.

2.2.1. Fixed 8-Rank Comparison (Per-Core Microarchitecture)

All three machines run 8 MPI ranks on 1 node. This controls for parallelism, isolating per-core architecture differences.

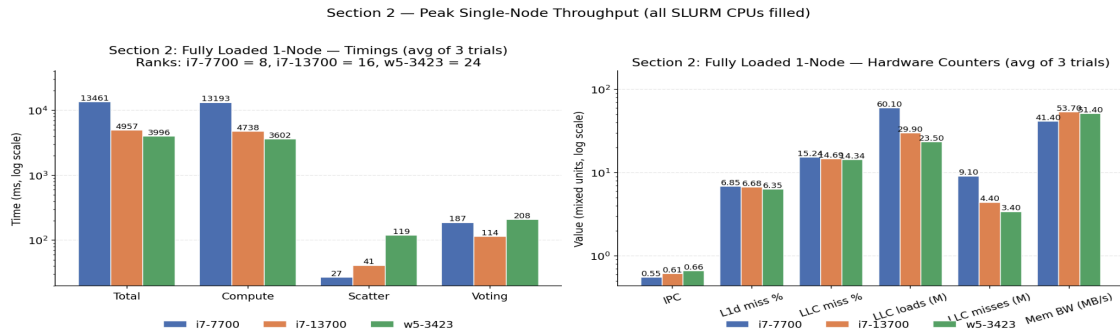


L1 miss rates are identical ($\sim 6.5\%$) because the access pattern is fixed by the binary. The w5-3423 has 32% fewer LLC loads (38.9M vs ~ 54.5 M) because each rank's ~ 2.5 MB working set mostly fits in its 2 MB L2, whereas both the i7-13700 (1.25 MB L2) and i7-7700 (256 KB L2) overflow to L3. Thus, the w5-3423 consequently needs the least DRAM bandwidth (38.3 MB/s). The i7-13700 compensates for lower IPC with its 5.2 GHz clock,

landing only 5.7% behind the w5-3423 in compute (9,416ms vs 8,909ms) despite a weaker per-clock pipeline. Voting, the serial phase on rank 0, isolates single-thread performance, and the ordering (i7-13700 139ms < w5-3423 151ms < i7-7700 183ms) confirms clock speed dominates once parallelism is removed. Together, cache hierarchy and microarchitecture generation explain the w5-3423's 1.48x total speedup over the i7-7700 at equal rank count.

2.2.2. Fully Loaded 1-Node (Peak Single-Node Throughput)

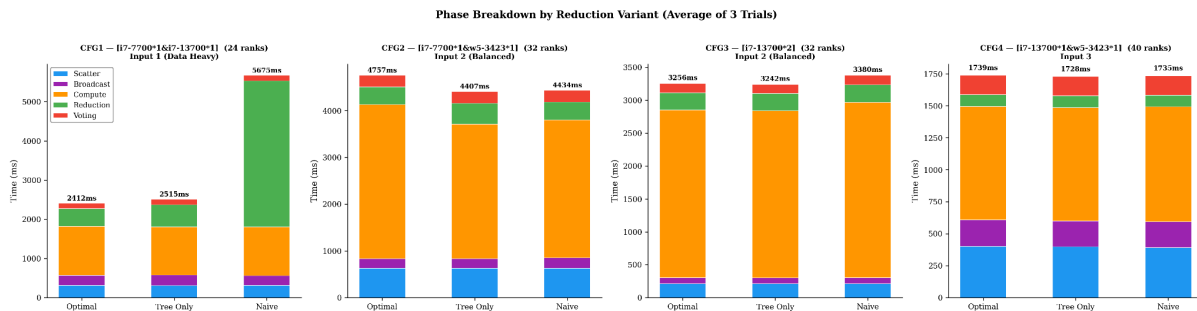
Each machine fills all its SLURM CPUs: i7-7700 = 8 ranks, i7-13700 = 16 ranks, w5-3423 = 24 ranks.



The w5-3423 finishes fastest at 3,996 ms, followed by the i7-13700 (4,957 ms) and i7-7700 (13,461 ms), with the computing phase dominating 90–98% of total time. The i7-13700's 2.71x speedup over the i7-7700 decomposes into 2x more ranks $\times$ 1.24x clock $\times$ 1.11x IPC = 2.75x, where the IPC gain comes from each rank's 1.25 MB working set fitting in its 1.25 MB L2 (cutting LLC loads 50% and misses 52%). The w5-3423's 3.37x speedup over the i7-7700 comes from 3x ranks $\times$ 1.20x IPC, since its 24 ranks shrink the per-rank working set to ~833 KB, well inside its 2 MB L2, cutting LLC loads 61% and misses 63% despite sharing the same 4.2 GHz clock. The w5-3423 also beats the i7-13700 by 1.24x total (1.32x compute) despite a 24% clock deficit, because its 2 MB L2 captures more of the working set than the i7-13700's 1.25 MB (21% fewer LLC loads, 8% higher IPC) and it has 1.5x more ranks. Across all three comparisons, the pattern is consistent: larger per-core L2 and more ranks compound to overcome raw clock-speed deficits, because fewer DRAM stalls directly raise sustained IPC.

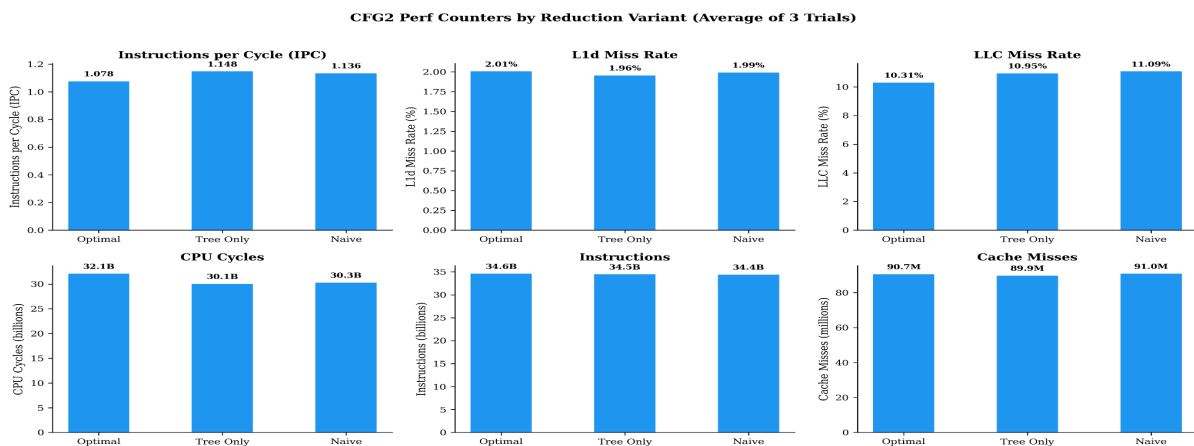
3. Optimisation Attempted: Tree-based and intra-node reduction

We benchmarked three reduction strategies, a naive linear gather, a flat binomial-tree reduction across MPI_COMM_WORLD and a two-stage intra-node-shared-memory plus cross-node tree reduction against the same four heterogeneous 2-node configurations used in the provided benchmarks, running each combination three times and recording per-phase timings on rank 0.



The graph above summarises per-phase timings (averaged over three trials) for the three reduction strategies across four hardware and input configurations. The benefit of the two-stage reduction is most visible in cfg1 (100,000 data points, 4,000 queries, i7-7700 + i7-13700), where the data-distribution path is taken and reduction dominates runtime. The naive linear gather spends 3,732ms in reduction alone, the tree variant drops this to 567ms (6.6× speedup, consistent with the $O(P) \rightarrow O(\log P)$ transition for $P=24$) and the optimal two-stage scheme reduces it further to 458ms by replacing intra-node MPI merges with shared-memory merges. End-to-end, optimal achieves a 2.35× speedup over naive (2,423ms vs 5,685ms).

In cfg2/cfg3/cfg4, the heuristic selects the query-distribution branch, where reduction degenerates to a single `MPI_Gather_v` shared by all variants, making the choice of reduction strategy effectively irrelevant. All three variants fall within 1–5% of each other in cfg3 and cfg4. The one anomaly is cfg2 (i7-7700 + w5-3423), where optimal runs ~10% slower in the compute phase. Hardware-counter profiling shows identical instruction counts (~34.5B) and cache miss rates across variants, but IPC drops from 1.14 to 1.08 with 30–35% higher iTLB misses, consistent with instruction-cache pressure from the unreached data-branch code on Kaby Lake's smaller 128-entry iTLB (see graph below). This effect vanishes on cfg3's i7-13700 nodes, whose larger iTLB absorbs the extra code footprint, confirming a binary-size rather than algorithmic cause.



4. Appendix

Raw data for the respective sections can be found here:

<https://drive.google.com/drive/folders/1gHr7zIsOSSLocRHRsdXljdDpRhWsxjf?usp=sharing>

Steps for replication of results can be found here:

<https://docs.google.com/document/d/1IRFcYfJcyN8pxWadZJEWrZhZPQ4fERCChruuyXHRo7k/edit?usp=sharing>